

Campus Recruitment Training — Phase 2 Post-Assessment Test

Shresth Shakrawar

Batch : B1

Section A — Spring Boot & Request Lifecycle

[25 marks]

Q1. Predict the output. What will print to the console when this Spring Boot application starts? Write the output in order and give a one-line reason for each.

[4M]

```
@Component
public class MyBean {
    public MyBean() {
        System.out.println("Constructor called");
    }

    @PostConstruct
    public void init() {
        System.out.println("Init called");
    }
}
```

Output in order:

1. Constructor called
2. Init called.

Reason: Spring first instantiates the bean using its constructor, and only after the bean is created and all dependencies are injected does it call the `@PostConstruct` method

Q2. The endpoint below compiles and runs but always returns an empty response body. Identify the mistake and write the corrected line only.

[3M]

```
@Controller
@RequestMapping("/api/users")
public class UserController {
    @Autowired UserService service;

    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        User u = service.findById(id);
        return new ResponseEntity<>(HttpStatus.OK);
    }
}
```

Correction: ResponseEntity should refer to user id.
return new ResponseEntity<>(u, HttpStatus.OK);

Q3. Write a Spring Boot REST endpoint POST /api/students that accepts a JSON body with fields name (must not be blank) and age (must be 18 or above). Return 400 Bad Request with an error message if validation fails, or 201 Created on success. Write the DTO class and the controller method.

[5M]

StudentDto.java

```
package com.practice.question3;

import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;

public class StudentDto {

    @NotBlank(message = "Name must not be blank")
    private String name;

    @Min(value = 18, message = "Age must be 18 or above")
    private int age;

    // Default constructor
    public StudentDto() {}

    // Getters and Setters
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }
}
```

StudentController.java

```
package com.practice.question3;
```

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;

import java.util.HashMap;
import java.util.Map;

@RestController
public class StudentController {

    // The POST endpoint
    @PostMapping("/api/students")
    public ResponseEntity<String> createStudent(@Valid @RequestBody
StudentDto student) {
    // If the code reaches here, validation passed.
        return new ResponseEntity<>("Student created successfully",
HttpStatus.CREATED);
    }

    // Catches validation errors and returns a 400 Bad Request with the
specific messages
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>>
handleValidationExceptions(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();

        // Extract the specific error messages defined in the DTO
        for (FieldError error : ex.getBindingResult().getFieldErrors()) {
            errors.put(error.getField(), error.getDefaultMessage());
        }

        return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST);
    }
}
```

Q4. What annotation should be added to the `transfer()` method below, and why? Write the annotation and explain in 2 lines what happens if it is missing.

[3M]

```
@Service
public class BankService {
    @Autowired AccountRepository repo;

    public void transfer(Long fromId, Long toId, double amount) {
        Account from = repo.findById(fromId).orElseThrow();
        Account to = repo.findById(toId).orElseThrow();
        from.setBalance(from.getBalance() - amount);
        to.setBalance(to.getBalance() + amount);
        repo.save(from);
        repo.save(to);
    }
}
```

`@Transactional` Annotation should be added here.

Without `@Transactional`, each `repo.save()` executes in its own separate database transaction, so if the second `save(to)` fails after the first `save(from)` succeeds, the debit occurs but the credit does not — leaving the database in a corrupt, inconsistent state. There is no automatic rollback, meaning money can be lost or created out of thin air due to a partial update.

Q5. Write a JPA entity class `Employee` with the following fields: `id` (auto-generated), `name` (String), `salary` (double), and a Many-to-One relationship to a `Department` entity. Show the `Department` entity with a One-to-Many list of employees. Use correct annotations on both sides.

[5M]

```
package com.practice.question5;

import jakarta.persistence.*;
import java.util.List;

@Entity
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // mappedBy refers to the "department" field in the Employee class
    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
    private List<Employee> employees;
}
```

```
// Getters and Setters  
}
```

```
package com.practice.question5;  
  
import jakarta.persistence.*;  
  
@Entity  
public class Employee {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    private String name;  
    private double salary;  
  
    // The @JoinColumn indicates the physical column in the database table  
    @ManyToOne  
    @JoinColumn(name = "department_id")  
    private Department department;  
  
    // Getters and Setters  
}
```

Q6. Look at the JWT filter below. (a) Explain in 2 lines what this filter does. (b) There is one bug — the token extracted from the header will always be invalid. Find it and write the corrected line.

[5M]

```
String header = req.getHeader("Authorization");  
if (header == null || !header.startsWith("Bearer ")) {  
    chain.doFilter(req, res);  
    return;  
}  
String token = header.substring(6); // extract token  
String username = jwtUtil.extractUsername(token);  
if (username != null) {  
    UsernamePasswordAuthenticationToken auth =  
        new UsernamePasswordAuthenticationToken(username, null, null);  
    SecurityContextHolder.getContext().setAuthentication(auth);  
}  
chain.doFilter(req, res);
```

Corrected Line String token = header.substring(7);

a) What this filter does: It intercepts incoming HTTP requests, extracts the JSON Web Token (JWT) from the "Authorization" header, validates the username, and if valid, populates the Spring `SecurityContextHolder` to authenticate the user for the current request.

b) The Bug & Corrected Line: The prefix "Bearer " (including the space) is exactly 7 characters long. Using `substring(6)` leaves a leading space attached to the token, which will cause the JWT parser to fail validation

Q7. Write a Spring Data JPA repository method using `@Query` to find all Product entities where price is less than a given value and category matches a given string. Use named parameters.

```
@Query("SELECT p FROM Product p WHERE p.price < :price AND p.category = :category") List<Product> findAffordableProductsByCategory( @Param("price") double price, @Param("category") String category );
```

Q8. Explain the N+1 query problem in JPA in your own words with a simple example. What is the most common cause? Write one line of code that fixes it in a `@Query` method using JOIN FETCH.

[4M]

The N+1 problem occurs when an ORM (like Hibernate) executes 1 query to retrieve a list of entities, and then executes an additional N queries to retrieve a lazily-loaded related entity for each item in that list. *Example:* You query 10 `Author` records (1 query). While looping through the authors, you call `author.getBooks()`. Hibernate executes a separate query to fetch the books for *each* of the 10 authors (10 queries). Total queries = 11.

Most Common Cause: Accessing a lazy association (`@OneToMany` or `@ManyToOne`) inside a loop over a collection of entities.

The Fix: Use `JOIN FETCH` to retrieve the parent and the related entities in a single SQL join query.

```
@Query("SELECT a FROM Author a JOIN FETCH a.books")
```

Q9. Write a SQL query to find the second highest salary from a table employees(id, name, salary). Use a subquery approach. Then write the same logic as a JPQL @Query method in a JPA repository.

[4M]

```
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

```
package com.practice.question9;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    @Query("SELECT MAX(e.salary) FROM Employee e WHERE e.salary < (SELECT MAX(e2.salary) FROM Employee e2)")
    Double findSecondHighestSalary();
}
```

Q10. Given a Student entity with fields name, email, and city, write FOUR Spring Data JPA derived query method names (no @Query) for the following: (a) find by city, (b) find by name and city, (c) find by email containing a keyword, (d) count students in a city. Write the method signatures only.

Derived queries allow you to create database queries simply by naming your Java methods correctly. You don't need any @Query annotations or SQL here.

(a) Find by city:

```
List<Student> findByCity(String city);
```

(b) Find by name and city:

```
List<Student> findByNameAndCity(String name, String city);
```

(c) Find by email containing a keyword:

```
List<Student> findByEmailContaining(String keyword);
```

(d) Count students in a city:

```
long countByCity(String city);
```

Q11. A developer runs the following code to load 10 orders and print their items. The [3M] Campus Recruitment Training — Phase 2 | Post-Assessment | Page 3 of 7 console shows 11 SQL queries instead of 1. What is this problem called? Write the line of code that causes it and the one-line fix.

```
List<Order> orders = orderRepo.findAll();
for (Order o : orders) {
    System.out.println(o.getItems().size());
}
```

Problem Name: The N+1 Query Problem.

Line of code that causes it: `System.out.println(o.getItems().size());`. (Because `getItems()` triggers a lazy-loading query for each individual order inside the loop).

One-line fix:

```
@Query("SELECT o FROM Order o JOIN FETCH o.items") List<Order>
findAllWithItems();
```

Q12. Write the application.yml configuration and the main class annotation needed to register a Spring Boot service named product-service with a Eureka server at <http://localhost:8761/eureka/>.

[4M]

Application.yml

```
spring:
  application:
    name: product-service
eureka:
  client:
    serviceUrl:
      defaultZone: http://localhost:8761/eureka/
```


Q13. What is a Circuit Breaker in microservices? Why is it needed? Write a Spring Boot method that calls an external payment-service using RestTemplate and applies a Resilience4j @CircuitBreaker with a fallback method that returns a default PaymentResponse object.

[5M]

A Circuit Breaker is a design pattern that prevents cascading failures in microservices. If Service A calls Service B, and Service B is down, Service A will normally wait until the connection times out. If thousands of requests are happening, Service A will quickly run out of threads and crash too. A Circuit Breaker detects the failures, "opens" the circuit to block further calls immediately, and provides a default "fallback" response so Service A stays alive.

```
package com.practice.question13;

import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

@Service
public class PaymentClientService {

    private final RestTemplate restTemplate;

    public PaymentClientService(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
    }

    @CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
    public PaymentResponse makePayment() {
        return restTemplate.getForObject("http://payment-service/api/pay",
PaymentResponse.class);
    }

    // The fallback method must have the exact same return type and take an
Exception parameter
    public PaymentResponse paymentFallback(Exception e) {
        return new PaymentResponse("FAILED_FALLBACK", 0.0);
    }
}
```

Q14. What is an API Gateway and what are 2 benefits of using one? Write a Spring Cloud Gateway application.yml that routes requests starting with /api/orders/** to order-service and /api/users/** to user-service. Use lb:// for load-balanced URIs.

[6M]

An API Gateway is a server that acts as the single entry point into a microservices architecture.

2 Benefits:

1. **Simplified Client Routing:** Frontend apps only need to know one URL, and the gateway handles routing the traffic to the correct underlying microservices.
2. **Centralized Security/Policies:** You can handle cross-cutting concerns like JWT authentication, CORS, and rate-limiting in one place rather than in every single microservice.

Application.yml

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service-route
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
        - id: user-service-route
          uri: lb://user-service
          predicates:
            - Path=/api/users/**
```

Q15. What is Service Discovery in microservices and why is it needed instead of hardcoding IP addresses? Name the tool you used in class. Write a RestTemplate bean configuration that enables it to resolve service names automatically.

[4M]

In the cloud, microservices scale up and down dynamically, meaning their IP addresses constantly change. Hardcoding IPs (like `http://192.168.1.5:8080`) will break your app. Service Discovery acts as a phonebook. Services register their current IP with it, and other services query it to find each other by name

Netflix Eureka is a commonly used tool for service discovery.

```
package com.practice.question15;
```

```
import org.springframework.cloud.client.loadbalancer.LoadBalanced;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestTemplate;

@Configuration
public class AppConfig {

    @Bean
    @LoadBalanced
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

Q16. You have a monolithic Spring Boot app with modules: `UserModule`, `OrderModule`, `ProductModule`. Your team wants to split it into microservices. (a) List 3 benefits of this split. (b) List 2 challenges. (c) Briefly explain how Eureka and an API Gateway work together so a React frontend can talk to all three services through a single URL.

[6M]

a) 3 Benefits:

1. **Independent Deployment:** You can update the `ProductModule` without having to redeploy and restart the entire application.
2. **Targeted Scaling:** If the `OrderModule` gets a lot of traffic, you can spin up 10 instances of it while keeping only 1 instance of the `UserModule`.
3. **Technology Flexibility:** You aren't locked into one tech stack; you could rewrite the `ProductModule` in Go or Python if it made sense, while keeping the others in Spring Boot.

b) 2 Challenges:

1. **Network Complexity & Latency:** Internal method calls are now network calls over HTTP, which are slower and can fail.
2. **Data Consistency:** Distributed transactions are hard. If an order fails, rolling back the payment and inventory across different databases is complex.

c) Eureka & API Gateway Interaction: The React frontend doesn't need to know where all three services live. It only sends requests to the **API Gateway's** single URL. The Gateway acts as a reverse proxy. When a request comes in (e.g., `/api/users`), the Gateway asks **Eureka** (the service registry) "What is the current IP address of the User Service?". Eureka replies with

the IP, and the Gateway forwards the React request to that exact location.

Q17. Write a JUnit 5 + Mockito unit test for the `getById()` method below. Your test should: [5M]
(a) use `@Mock` and `@InjectMocks`, (b) stub the repository to return a sample Product, and
(c) assert that the returned product name equals "Laptop".

```
@Service
public class ProductService {
    @Autowired ProductRepository repo;

    public Product getById(Long id) {
        return repo.findById(id).orElseThrow();
    }
}
```

```
package com.practice.testing;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import java.util.Optional;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.when;

@ExtendWith(MockitoExtension.class)
class ProductServiceTest {

    @Mock
    private ProductRepository repo; // Creates a fake repository

    @InjectMocks
    private ProductService service; // Injects the fake repo into the real
    service

    @Test
    void testGetById() {
        // 1. Arrange (Setup the mock)
        Product mockProduct = new Product();
        mockProduct.setId(1L);
    }
}
```

```

        mockProduct.setName("Laptop");

        when(repo.findById(1L)).thenReturn(Optional.of(mockProduct));

        // 2. Act (Call the actual method)
        Product result = service.getById(1L);

        // 3. Assert (Verify the result)
        assertEquals("Laptop", result.getName());
    }
}

```

Q18. What is the difference between `@Mock` (Mockito) and `@MockBean` (Spring Boot Test)? When would you use each? Write one example of a test class declaration that uses `@MockBean` correctly with `@SpringBootTest`.

[4M]

- **@Mock:** A pure Mockito annotation. It creates a lightweight, isolated mock object instantly. It does **not** load the Spring Application Context. Use this for pure, fast unit tests.
- **@MockBean:** A Spring Boot Test annotation. It creates a mock and explicitly replaces the real bean *inside the Spring Application Context*. Use this for integration tests (like `@WebMvcTest` or `@SpringBootTest`) where you need Spring to wire things up, but still want to fake a specific layer (like the database).

```

@SpringBootTest
class UserControllerIntegrationTest {

    // Spring will load the whole app context, but replace the real
    // UserService with this mock
    @MockBean
    private UserService userService;

    @Autowired
    private UserController userController;

    // tests..
}

```

Q19. Write a Dockerfile for a Spring Boot application. The JAR file is named app.jar and is inside the target/ folder. Use eclipse-temurin:21-jre as the base image. The app runs on port 8080. Then write the docker build and docker run commands to build the image as myapp:latest and run it.

[5M]

```
FROM eclipse-temurin:21-jre
COPY target/app.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Terminal Commands:

1. **Build the image:** `docker build -t myapp:latest .`
 2. **Run the container:** `docker run -p 8080:8080 myapp:latest.`
-

Q20. Write a docker-compose.yml with two services: (a) db — using the postgres:15 image with environment variables for DB name, user, and password, and (b) app — using the myapp:latest image, mapping port 8080, connecting to the db service, and starting only after db starts using depends_on.

[6M]

docker-compose.yml

```
version: '3.8'
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
    ports:
      - "5432:5432"

  app:
    image: myapp:latest
    ports:
```

```
- "8080:8080"
depends_on:
  - db
environment:
  # Tell the app how to find the database container
  SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/mydb
  SPRING_DATASOURCE_USERNAME: user
  SPRING_DATASOURCE_PASSWORD: password
```

Q21. Write a JavaScript async function `loadUsers()` that fetches data from `GET /api/users` using the `fetch` API, checks if the response is successful (`response.ok`), and either renders a list of user names into a `<ul id="user-list">` element or shows an error message in a `<p id="error">` element.

[5M]

```
const data = await response.json();
});
```

```
async function loadUsers() {
  try {
    const response = await fetch('/api/users');

    if (!response.ok) {
      throw new Error(`HTTP error! Status: ${response.status}`);
    }

    const data = await response.json();
    const userList = document.getElementById('user-list');

    // Clear the list before appending to avoid duplicates
    userList.innerHTML = '';

    data.forEach(user => {
      const li = document.createElement('li');
      li.textContent = user.name;
      userList.appendChild(li);
    });
  } catch (error) {
    // Handle error
  }
}
```

```

    });

    } catch (error) {
        document.getElementById('error').textContent = 'Error loading
users: ' + error.message;
    }
}

```

Q22. The React component below has a bug that causes it to fetch data in an infinite loop. [5M]
(a) Identify exactly what is wrong. (b) Fix it. (c) Add a simple loading state that shows 'Loading...' while data is being fetched.

```

function UserList() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    fetch('/api/users')
      .then(res => res.json())
      .then(data => setUsers(data));
  }); // <-- notice anything?

  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

```

The `useEffect` hook triggers a side effect (fetching data). It lacks a dependency array (a `[]` as the second argument), it runs after *every single render*. When the fetch completes, it calls `setUsers(data)`. Updating state forces a re-render. The re-render triggers `useEffect` again, causing another fetch, another state update, another re-render... an infinite loop.

Fixed Code:

```

import React, { useState, useEffect } from 'react';

function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true); // Added loading state

  useEffect(() => {
    fetch('/api/users')
      .then(res => res.json())
      .then(data => {
        setUsers(data);
        setLoading(false); // Data arrived, turn off loading
      });
  }, []);

  return <div>
    {loading ? <div>Loading...</div> : <ul>
      {users.map(u => <li key={u.id}>{u.name}</li>)}
    </ul>}
  </div>;
}

```



```
    })
    .catch(err => {
      console.error("Failed to fetch", err);
      setLoading(false);
    });
  }, []); // <-- FIX: The empty dependency array means this runs ONLY
ONCE on mount

  if (loading) {
    return <p>Loading...</p>;
  }

  return (
    <ul>
      {users.map(u => (
        <li key={u.id}>{u.name}</li>
      ))}
    </ul>
  );
}
```
